

Graphics and Qt6 on Yale HPC Milgram: A Guide for MEG/EEG Researchers

The Challenge: Why Graphics Don't "Just Work" on HPC

If you've tried running MNE-Python visualizations on Milgram and encountered cryptic errors like "qt.qpa.xcb: could not connect to display" or simply "Killed", you've stumbled into one of the most frustrating aspects of scientific computing: the graphics backend problem.

Here's what's happening: Your local computer has a physical display, graphics drivers, and a windowing system that applications can directly access. When you run `python visualize_data.py` on your laptop, Qt6 (the graphics framework used by MNE) simply asks your operating system to create a window, and it appears.

Milgram, like most HPC systems, has no physical displays attached to its compute nodes. You're connecting remotely, often through multiple network hops, and asking a server sitting in a data center to somehow display graphics on your local screen. This requires a complex chain of software translations that can break at any point.

The Technical Stack: What's Actually Involved

When you run MNE-Python with Qt6 graphics on Milgram through Open OnDemand (OOD), here's what needs to happen:

1. **Python/MNE** generates visualization commands
2. **Qt6** tries to render these using a graphics backend
3. **PyQt6** provides Python bindings to Qt6
4. **X11/XCB** protocol attempts to forward graphics over the network
5. **OOD's display server** captures this and routes it to your browser
6. **Your browser** finally displays the image

Each layer has its own configuration requirements, environment variables, and potential failure points.

Common Failure Modes and What They Mean

"qt.qpa.xcb: could not connect to display"

Qt6 can't find a display server. The `QT_QPA_PLATFORM` environment variable isn't set correctly, or the `DISPLAY` variable is missing.

"Killed" with no explanation

Often means memory exhaustion. Qt6's rendering buffers, combined with large MEG datasets, can exceed memory limits. Check `dmesg` output for OOM (out-of-memory) killer messages.

"GLX: Failed to create context"

OpenGL acceleration isn't available. This happens when Qt6 tries to use hardware acceleration that doesn't exist on compute nodes.

Silent hangs during visualization

Usually indicates Qt6 is waiting for a display server response that will never come. The backend is misconfigured for the environment.

Memory Considerations: The Hidden Graphics Cost

What many researchers don't realize is that graphics operations can dramatically increase memory usage. When MNE creates an interactive plot of your MEG data:

1. The raw data exists in memory (potentially several GB)
2. Filtered copies might be created for display (doubling memory use)
3. Qt6 creates rendering buffers (hundreds of MB to several GB)
4. Graphics textures and caches add overhead

A dataset that fits comfortably in 32GB for analysis might need 64GB+ when generating interactive visualizations.

The Solution: Configuring Qt6 for the MEG Pipeline

The MEG pipeline requires interactive graphics for quality control and artifact review. To make this work on Milgram, you need to properly configure the Qt6 backend environment. This requires setting multiple environment variables that tell Qt6 how to find displays, plugins, and libraries.

The Complete Setup Script for MEG Pipeline on Milgram

Below is a comprehensive shell script that sets up the environment for running the MEG pipeline with Qt6 graphics on Milgram. Save this as `setup_mne_graphics.sh`:

```
#!/usr/bin/env bash
#
=====
====
# MNE-Python Qt6 Graphics Setup for Yale HPC Milgram MEG Pipeline
#
# This script configures the environment for running MNE-Python
# visualizations
# on Milgram's Open OnDemand system. It handles the complex graphics
# backend
# configuration required for Qt6 to work in a remote HPC environment.
#
# Usage:
#   source ./setup_mne_graphics.sh
#   python your_meg_pipeline_script.py
#
# Last Updated: November 2025
#
=====
```

```

=====

set -e # Exit on any error

echo
"=====
echo "   MEG Pipeline Graphics Environment Setup for Milgram"
echo
"=====
echo ""

# -----
# STEP 1: Initialize Conda
# Milgram uses a system-wide conda installation that must be properly
# sourced
# -----
echo "📦 Step 1: Initializing Conda environment..."

CONDA_PATH="/gpfs/milgram/apps/avx2/software/miniconda/24.11.3"

if [ -f "$CONDA_PATH/etc/profile.d/conda.sh" ]; then
    source "$CONDA_PATH/etc/profile.d/conda.sh"
    echo "   ✓ Found Conda at: $CONDA_PATH"
else
    echo "   ✗ ERROR: Could not find Conda installation"
    echo "   Expected location: $CONDA_PATH"
    echo "   Please contact HPC support if this path has changed"
    exit 1
fi

# -----
# STEP 2: Activate MNE Environment
# This assumes you've already created a conda environment called 'mne'
# with MNE-Python and PyQt6 installed for the MEG pipeline
# -----
echo "🧠 Step 2: Activating MNE environment..."

if conda activate mne 2>/dev/null; then
    echo "   ✓ Activated 'mne' environment"
    echo "   Python: $(which python)"
    echo "   Version: $(python --version)"
else
    echo "   ✗ ERROR: Could not activate 'mne' environment"
    echo ""
    echo "   The MEG pipeline requires an environment named 'mne' with:"
    echo "   - MNE-Python"
    echo "   - PyQt6"
    echo "   - NumPy, SciPy, Matplotlib"
    echo ""
    echo "   Create it with:"

```

```

    echo "    conda create -n mne python=3.10"
    echo "    conda activate mne"
    echo "    pip install mne PyQt6 matplotlib"
    exit 1
fi

# -----
# STEP 3: Configure Qt6 Display Backend
# This tells Qt6 how to handle graphics in a headless environment
# -----
echo "🖥️ Step 3: Configuring Qt6 display backend..."

# XCB is the X11 backend that can forward graphics over network
# This is required for the MEG pipeline's interactive displays
export QT_QPA_PLATFORM=xcb
echo "    → QT_QPA_PLATFORM=$QT_QPA_PLATFORM"

# Disable GPU acceleration (not available on compute nodes)
# This prevents OpenGL errors that would crash the pipeline
export QT_QUICK_BACKEND=software
export LIBGL_ALWAYS_SOFTWARE=1
echo "    → Software rendering enabled (no GPU acceleration)"

# Set matplotlib to use Qt backend for MEG pipeline compatibility
export MPLBACKEND=QtAgg
export QT_API=pyqt6
echo "    → Matplotlib backend: $MPLBACKEND"

# -----
# STEP 4: Configure PyQt6 Plugin and Library Paths
# Qt6 needs to know where to find its plugins and libraries
# Without this, the MEG pipeline will fail with plugin errors
# -----
echo "🔧 Step 4: Setting up PyQt6 plugin paths..."

# Find where PyQt6 is installed
PYQT6_BASE=$(python -c "import PyQt6, os;
print(os.path.dirname(PyQt6.__file__))" 2>/dev/null)

if [ -z "$PYQT6_BASE" ]; then
    echo "    ✖ ERROR: PyQt6 not found in current environment"
    echo "    The MEG pipeline requires PyQt6 for visualizations"
    echo "    Install with: pip install PyQt6"
    exit 1
fi

echo "    → Found PyQt6 at: $PYQT6_BASE"

# Tell Qt where to find plugins (critical for display backends)
export QT_PLUGIN_PATH="$PYQT6_BASE/Qt6/plugins"

```

```

export QT_QPA_PLATFORM_PLUGIN_PATH="$PYQT6_BASE/Qt6/plugins/platforms"

# Add Qt libraries to library path - required for the pipeline to find Qt6
components
export
LD_LIBRARY_PATH="$PYQT6_BASE/Qt6/lib:$CONDA_PREFIX/lib:${LD_LIBRARY_PATH}"

echo "    → Plugin path configured"

# -----
# STEP 5: Memory and Performance Optimizations
# These settings help prevent the "Killed" error when processing large MEG
files
# -----
echo "> Step 5: Applying MEG pipeline optimizations..."

# Limit OpenMP threads to reduce memory overhead
# The MEG pipeline can use excessive memory with too many threads
export OMP_NUM_THREADS=4
export MKL_NUM_THREADS=4
echo "    → Limited parallel threads to 4"

# Disable Qt debugging output (saves memory)
export QT_LOGGING_RULES="*.debug=false"
echo "    → Disabled Qt debug logging"

# Use basic render loop (more stable in remote environments)
# This prevents rendering issues in the MEG pipeline's interactive plots
export QSG_RENDER_LOOP=basic
echo "    → Using basic Qt render loop"

# Configure MNE for memory efficiency
export MNE_USE_NUMBA='false' # Reduces memory overhead
echo "    → Disabled Numba JIT compilation for memory efficiency"

# -----
# STEP 6: Verify Display Configuration
# Check if we have a valid display connection for the pipeline
# -----
echo "🔍 Step 6: Verifying display configuration..."

if [ -z "$DISPLAY" ]; then
    echo "    ⚠ WARNING: DISPLAY variable not set"
    echo "    Attempting to set DISPLAY=:0"
    export DISPLAY=:0
fi

echo "    → DISPLAY=$DISPLAY"

# Test if we can actually connect to the display

```

```

if timeout 2 python -c "from PyQt6.QtWidgets import QApplication; app =
QApplication([])" 2>/dev/null; then
    echo "    ✓ Qt6 can connect to display – MEG pipeline graphics should
work"
else
    echo "    ⚠ WARNING: Qt6 cannot connect to display"
    echo "    The MEG pipeline may not be able to show interactive plots"
    echo "    Check your OOD session display settings"
fi

# -----
# STEP 7: Show Memory Limits
# Critical for understanding why the MEG pipeline might be killed
# -----
# -----

echo "📁 Step 7: Checking memory limits for MEG data processing..."

# Show current memory limit
MEMORY_LIMIT=$(ulimit -m)
if [ "$MEMORY_LIMIT" = "unlimited" ]; then
    echo "    → Memory limit: unlimited"
else
    MEMORY_KB=$MEMORY_LIMIT
    MEMORY_GB=$(echo "scale=2; $MEMORY_KB / 1024 / 1024" | bc)
    echo "    → Memory limit: ${MEMORY_GB} GB"
    echo "    ⚠ WARNING: MEG pipeline will be killed if it exceeds this
limit"
    echo "    Large MEG files with filtering may need 64GB+ of memory"
fi

# Check available memory
if command -v free &> /dev/null; then
    AVAILABLE_MEM=$(free -h | awk '/^Mem:/ {print $7}')
    echo "    → Available memory: $AVAILABLE_MEM"
fi

# -----
# SUCCESS: Environment Ready for MEG Pipeline
# -----
# -----

echo ""
echo "
=====
echo "✅ MEG Pipeline Environment Configured Successfully!"
echo ""
echo "You can now run your MEG pipeline scripts:"
echo "    python your_meg_pipeline_script.py"
echo ""
echo "Common MEG Pipeline Issues and Solutions:"
echo ""
echo "• 'Killed' during filtering or visualization:"
echo "    → Your data exceeded memory limits (common with hour-long

```

```
recordings)"
echo " → Request more memory in your OOD session (64GB+ recommended)"
echo ""
echo "• Graphics windows don't appear:"
echo " → Check that your OOD session has desktop enabled"
echo " → Verify DISPLAY variable matches your session"
echo ""
echo "• Qt platform plugin errors:"
echo " → Run: source ./setup_mne_graphics.sh (don't use ./)"
echo " → Ensure PyQt6 is installed: pip install PyQt6"
echo ""
echo "• Memory optimization for large MEG files:"
echo " Add to your pipeline script:"
echo "     raw = mne.io.read_raw_fif(file, preload=False)"
echo "     raw.filter(1.0, 40.0, n_jobs=1) # Use n_jobs=1 to save memory"
echo
"====="
```